SOFTWARE DEVELOPER INTERNSHIP ROUND — TECHNICAL DELIVERABLE

HackKro Internship Management System

Enterprise Full-Stack SaaS Platform for Milestone Tracking, GitHub Deliverables, Atomic Evaluation & Automated Communications

CANDIDATE / ENGINEER

Neeraj

SUBMISSION ROUND

Unstop Developer Assessment

GITHUB REPOSITORY

github.com/neerajcoder1/hackkro-internship-management

PRODUCTION FRONTEND

hackkro-internship-management.vercel.app

BACKEND REST API

hackkro-internship-management.onrender.com

VERIFIED TEST STATUS

● 46 / 46 Passing (100% Pass Rate)

Technology: React 18, Vite, Tailwind CSS, Node.js, Express, MongoDB Atlas, Resend

Date: November 2026

1. Executive Summary

The **HackKro Internship Management System** is an enterprise-grade SaaS application designed to streamline, track, and evaluate technical internships at scale. Built with a decoupled client-server architecture, the platform delivers role-tailored workspaces for enrolled interns and program administrators. Interns manage daily and weekly project deliverables, submit public GitHub repositories, track cumulative scores, and receive structured feedback. Administrators monitor intern rosters, assign milestones, review live submissions, allocate verified points, and trigger automated transactional notifications via Resend.

2. Problem Statement

Managing cohort-based engineering internships across distributed teams faces five core operational challenges:

- **Fragmented Communication:** Assignments, submissions, and feedback scattered across Google Sheets, email threads, and Discord channels cause tracking errors and missed deadlines.
- **Submission Verification Deficits:** Lack of standardized GitHub URL validation and pending-state concurrency guards leads to submission bottlenecks.
- **Scoring Inconsistencies & Race Conditions:** Manual grading frequently results in duplicate point awards or scores exceeding task limits.
- **Delayed Feedback Cycles:** Interns wait days without visibility into review status or milestones.
- **Privilege Escalation Risks:** Ad-hoc portals often lack strict backend role-based access control (RBAC), leaving administrative routes vulnerable to tampering.

3. Project Objectives

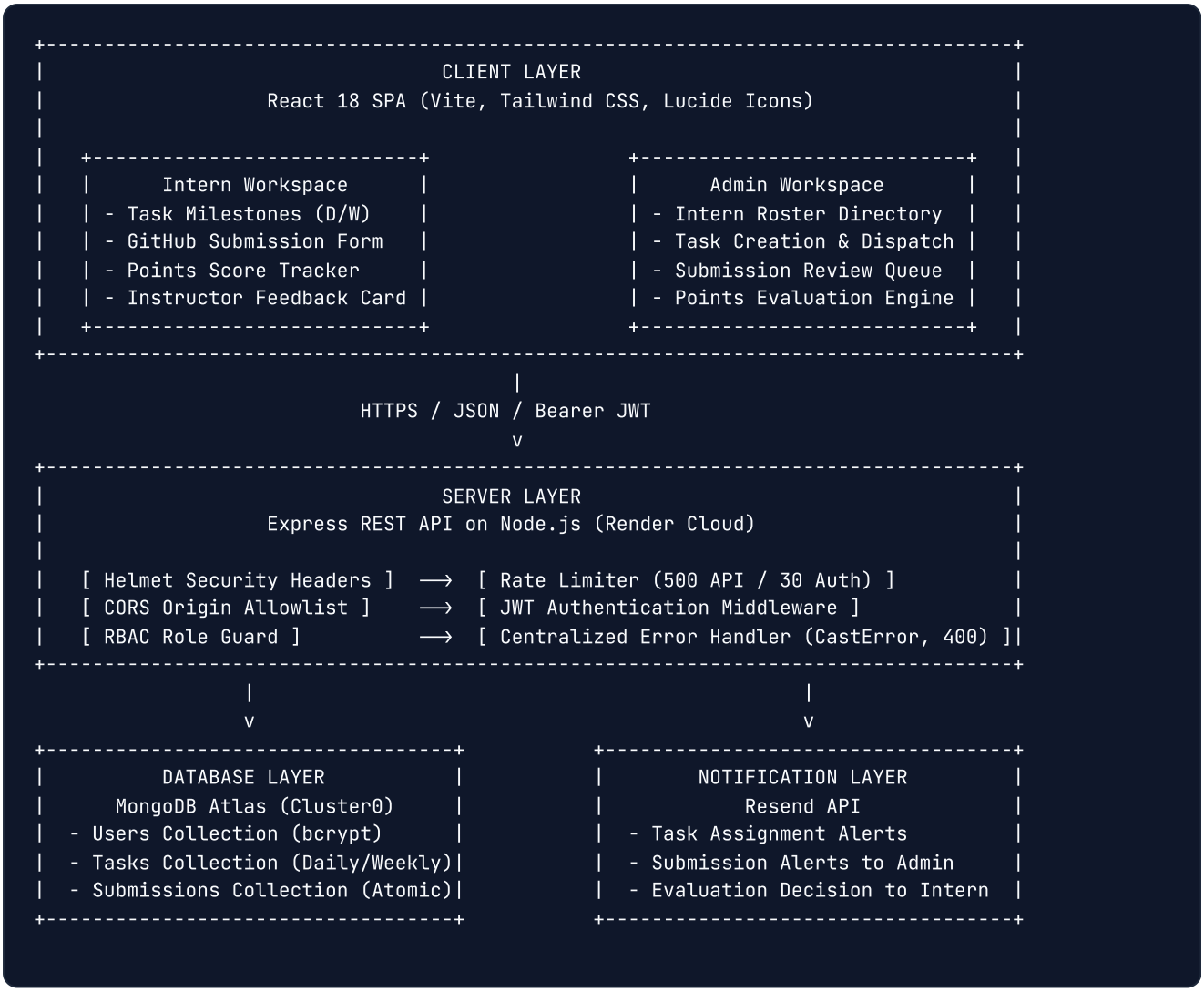
- **Centralized Milestone Delegation:** Create daily and weekly assignments with deadlines, markdown instructions, and maximum score caps.
- **Strict Role-Based Access Control:** Ensure public registration strictly yields the `intern` role, isolating admin endpoints behind JWT cryptographic verification.
- **Validated Deliverable Pipeline:** Enforce public GitHub repository URL validation and concurrency locks on pending evaluations.
- **Atomic Points Evaluation:** Prevent double-point awards using database-level atomic conditional updates.
- **Automated Lifecycle Notifications:** Deliver responsive, entity-escaped HTML transactional emails via Resend for task assignments, submissions, and evaluation decisions.

- **Enterprise SaaS UI/UX:** Implement a modern white-and-blue interface inspired by Linear and Stripe, with responsive layouts, modal dialogs, and real-time statistics.

4. Technology Stack

Layer	Technologies	Role & Architecture Benefit
Frontend	React 18.3, Vite 6, Tailwind CSS 3, Lucide Icons	Fast ESBuild bundling, atomic utility styling, client-side routing with SPA rewrite support.
Backend API	Node.js 20+, Express 4.21 (ESM)	Decoupled REST API, centralized error mapping, rate-limiting and security headers.
Database	MongoDB Atlas, Mongoose 8.9 ODM	Cloud-hosted document database, pre-save bcrypt hashing, atomic conditional updates.
Authentication	JWT (JSON Web Tokens), bcryptjs (10 rounds)	Stateless HMAC-SHA256 signatures, bearer token Authorization header, password stripping.
Email Service	Resend API SDK	Transactional email delivery with entity-escaped HTML templates and in-memory deduplication.
Cloud Hosting	Vercel (Frontend), Render (Backend)	Edge CDN caching, zero-downtime health probing, automatic GitHub-linked CI/CD deploys.

5. System Architecture


CLIENT LAYER
React 18 SPA (Vite, Tailwind CSS, Lucide Icons)
Intern Workspace
- Task Milestones (D/W)
- GitHub Submission Form
- Points Score Tracker
- Instructor Feedback Card
Admin Workspace
- Intern Roster Directory
- Task Creation & Dispatch
- Submission Review Queue
- Points Evaluation Engine
HTTPS / JSON / Bearer JWT
SERVER LAYER
Express REST API on Node.js (Render Cloud)
[Helmet Security Headers] ⟶ [Rate Limiter (500 API / 30 Auth)]
[CORS Origin Allowlist] ⟶ [JWT Authentication Middleware]
[RBAC Role Guard] ⟶ [Centralized Error Handler (CastError, 400)]
DATABASE LAYER
MongoDB Atlas (Cluster0)
- Users Collection (bcrypt)
- Tasks Collection (Daily/Weekly)
- Submissions Collection (Atomic)
NOTIFICATION LAYER
Resend API
- Task Assignment Alerts
- Submission Alerts to Admin
- Evaluation Decision to Intern

6. Workspace Functionality

6.1. Intern Workspace (`/intern`)

The Intern Portal is tailored for day-to-day milestone execution and transparent progress tracking:

- **Overview Statistics:** Real-time KPIs displaying Total Points, Completed Tasks, Active Milestones, and Cohort Completion Rate (%).
- **Daily & Weekly Task Listings:** Filterable tabs separating quick daily deliverables from multi-day weekly milestones.
- **Deliverable Submission Form:** Validates public GitHub URLs (`https://github.com/owner/repo`), enforces input sanitization, and captures student implementation notes.
- **Review Status Indicators:** Real-time status tags (`Pending`, `Submitted`, `Approved`, `Revision Requested`).
- **Feedback & Points Ledger:** Displays instructor evaluation notes alongside awarded score breakdowns (`85 / 100 Points`).
- **Profile & Membership Details:** Visualizes candidate account creation timestamp, verified role badge, and unique MongoDB Member ID.

6.2. Administrator Workspace (`/admin`)

The Administrator Command Center provides complete program oversight:

- **Program Overview Dashboard:** Aggregates total enrolled interns, total assigned tasks, deliverables awaiting review, and cumulative points awarded.
- **Intern Directory:** Searchable roster showing student names, email addresses, enrollment dates, and completed task tallies.
- **Task Creation & Delegation Engine:** Modal interface to create assignments, define deadlines, set max points, and designate assigned interns.
- **Submission Review Queue:** Real-time review queue highlighting pending student solutions with direct links to external GitHub repositories.
- **Evaluation & Scoring Engine:** Review modal to approve/reject submissions, allocate points (validated between 0 and task maxPoints), and provide qualitative feedback.

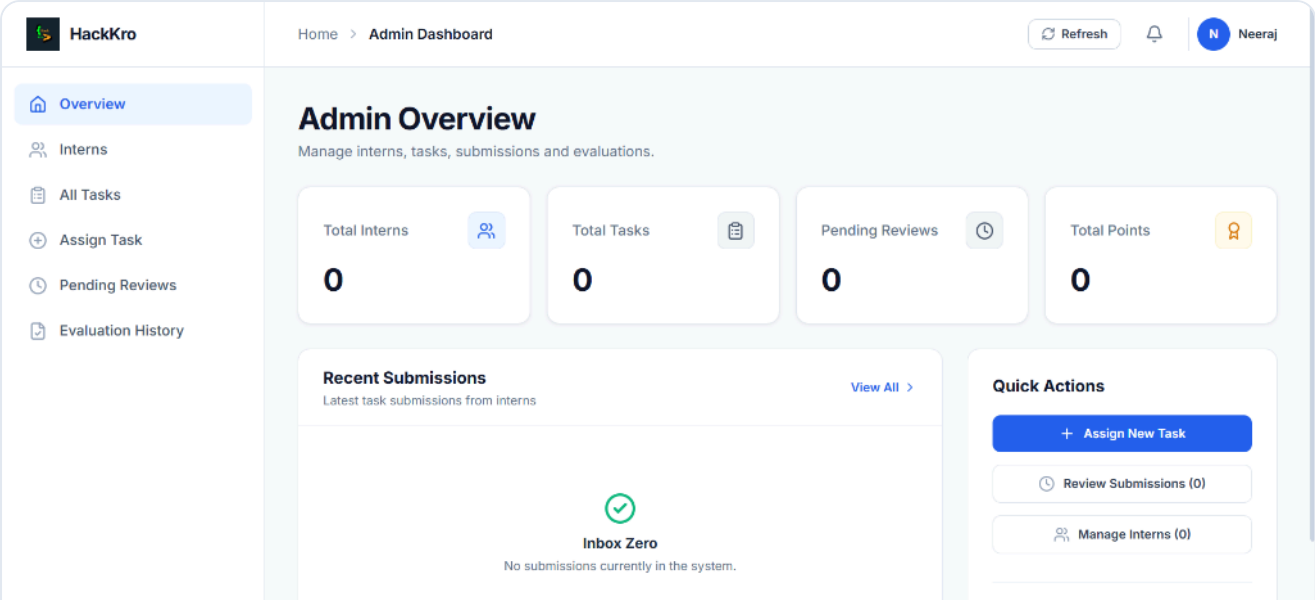
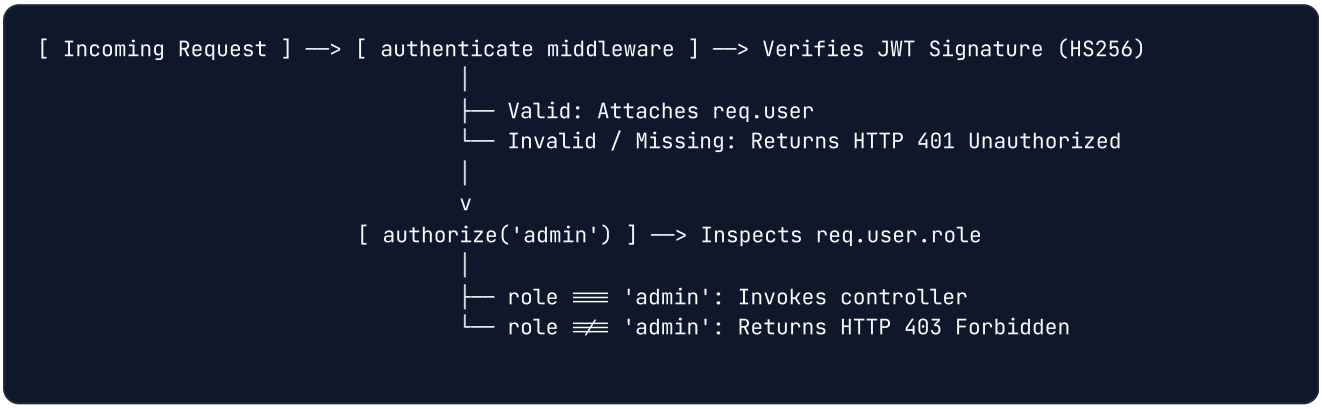


Figure 1: Live Production Administrator Command Center displaying KPIs, Quick Actions, and Review Queue.

7. Authentication & Role-Based Access Control (RBAC)

The system implements stateless JWT authentication stored in client `localStorage` and transmitted via the standard `Authorization: Bearer <token>` header.



8. Database Schemas & Data Integrity

Collection	Field	Type & Validation	Description & Indexing
Users (`users`)	`name`	String, required, minlength: 2	Candidate full name.
	`email`	String, required, unique, lowercase	Unique user identifier with B-tree index.
	`password`	String, required, select: false	bcrypt hash (10 salt rounds); omitted from queries.
	`role`	String, enum: ['intern', 'admin']	Access role, default: 'intern'.
Tasks (`tasks`)	`title` / `description`	String, required, trim	Milestone description and instructions.
	`taskType`	String, enum: ['daily', 'weekly']	Milestone cadence classification.
	`assignedTo` / `assignedBy`	ObjectId (ref: User), required	Indexed foreign keys to intern and admin users.

Collection	Field	Type & Validation	Description & Indexing
	`deadline` / `maxPoints`	Date / Number (min: 1)	Deadline timestamp and maximum allowable score.
Submissions (`submissions`)	`task` / `intern`	ObjectId (ref: Task, User)	Indexed relational foreign keys.
	`githubUrl`	String, required, regex validated	Public repository URL (`https://github.com/owner/repo`).
	`status`	String, enum: ['submitted', 'approved', 'rejected']	Review workflow lifecycle state.
	`pointsAwarded` / `isEvaluated`	Number (min: 0) / Boolean	Awarded points and atomic concurrency guard flag.

9. REST API Specification

Method	Endpoint	Access Level	Request Body / Query	Response Codes
`GET`	`/api/health`	Public	None	`200 OK`
`POST`	`/api/auth/register`	Public	`{ name, email, password }`	`201 Created`, 400, 409
`POST`	`/api/auth/login`	Public	`{ email, password }`	`200 OK`, 400, 401
`GET`	`/api/auth/me`	Authenticated	Bearer Token	`200 OK`, 401
`GET`	`/api/auth/interns`	Admin Only	Bearer Token	`200 OK`, 401, 403
`POST`	`/api/auth/register-admin`	Master Key	`x-admin-key` header	`201 Created`, 400, 403
`GET`	`/api/tasks/my-tasks`	Intern Only	Bearer Token	`200 OK`, 401
`POST`	`/api/tasks`	Admin Only	`{ title, taskType, assignedTo, deadline, maxPoints }`	`201 Created`, 400, 403
`GET`	`/api/tasks`	Admin Only	Filter query params	`200 OK`, 401, 403
`POST`	`/api/submissions`	Intern Only	`{ taskId, githubUrl, submissionNotes }`	`201 Created`, 400, 403
`GET`	`/api/submissions/my-submissions`	Intern Only	Bearer Token	`200 OK`, 401
`GET`	`/api/submissions/my-points`	Intern Only	Bearer Token	`200 OK`, 401

Method	Endpoint	Access Level	Request Body / Query	Response Codes
`PATCH`	`/api/submissions/:id/review`	Admin Only	`{ status, pointsAwarded, feedback }`	200 OK, 400, 403, 404

10. Automated Email Notification Workflows (Resend)

The notification service automates three critical milestones without blocking database transactions:

- **Task Assignment Workflow:** Dispatches to intern on task creation; details title, deadline, max score, and dashboard CTA.
- **Task Submission Workflow:** Dispatches to admin inbox (`ADMIN_NOTIFICATION_EMAIL`); includes student name, GitHub URL, notes, and direct review queue link.
- **Task Evaluation Decision:** Dispatches to intern on approval/rejection; reports decision badge (`APPROVED` or `REVISION REQUESTED`), score awarded (`85/100`), and qualitative instructor feedback.
- **Security & Sanitization:** All user strings are entity-escaped via `escapeHtml()` to prevent HTML injection/XSS in email clients.

11. Security Implementation

- **Atomic Evaluation Guard:** Replaced vulnerable read-then-save logic with an atomic MongoDB condition: `Submission.findOneAndUpdate({ _id: id, status: { $ne: 'approved' } }, ...)`. Concurrent duplicate evaluation requests are atomically blocked.
- **HTTP Security Headers:** Integrated `helmet()` enforcing `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, and HSTS.
- **Rate Limiting:** Configured `express-rate-limit` enforcing 500 requests per 15 min globally, and 30 attempts per 15 min on authentication routes.
- **Mongoose CastError Handling:** Centralized error middleware intercepts invalid MongoDB ObjectIds and returns `400 Bad Request` instead of unhandled 500 crashes.
- **Production Error Masking:** Operates under `NODE_ENV=production`, suppressing internal stack traces in API error responses.

12. Automated Testing Verification (46 / 46 Passing)

Test Suite	Tests	Passed	Failed	Key Assertions Covered
Phase 2: Authentication & RBAC	9	9 (100%)	0	Registration, duplicate email 409, bcrypt password verification, JWT generation, intern admin lockout 403.
Phase 5: Resend Email Workflows	5	5 (100%)	0	Task assignment dispatch, submission alert, evaluation alert, deduplication key skips, resilience on missing params.
Phase 7: Security Hardening & Edge Cases	21	21 (100%)	0	Missing/invalid/expired JWT, wrong password masking, rate limit headers, cross-intern submission 403, invalid task ID 400, duplicate pending submission 400, points > maxPoints 400, negative points 400, duplicate point prevention.
Phase 3: Task & Evaluation Lifecycle	11	11 (100%)	0	Admin task creation, intern task listing, GitHub submission creation, points calculation aggregation, submission history retrieval.
TOTAL VERIFIED RESULTS	**46**	46 (100%)	**0**	**Full automated suite execution time: ~13.2s**

13. Deployment Architecture

The system is deployed across an enterprise cloud topology:

- **Frontend (Vercel):** Hosted at `https://hackkro-internship-management.vercel.app` with `vercel.json` rewrite rules forwarding all subpaths (`/(.*)`) to `/index.html` for seamless SPA browser reloads.
- **Backend (Render):** Hosted at `https://hackkro-internship-management.onrender.com` running Node.js in production mode with dynamic port binding and zero-downtime health probes (`/api/health`).
- **Database (MongoDB Atlas):** Multi-region replica set cluster with SSL/TLS encryption and IP access allowlisting.

14. Application User Interface

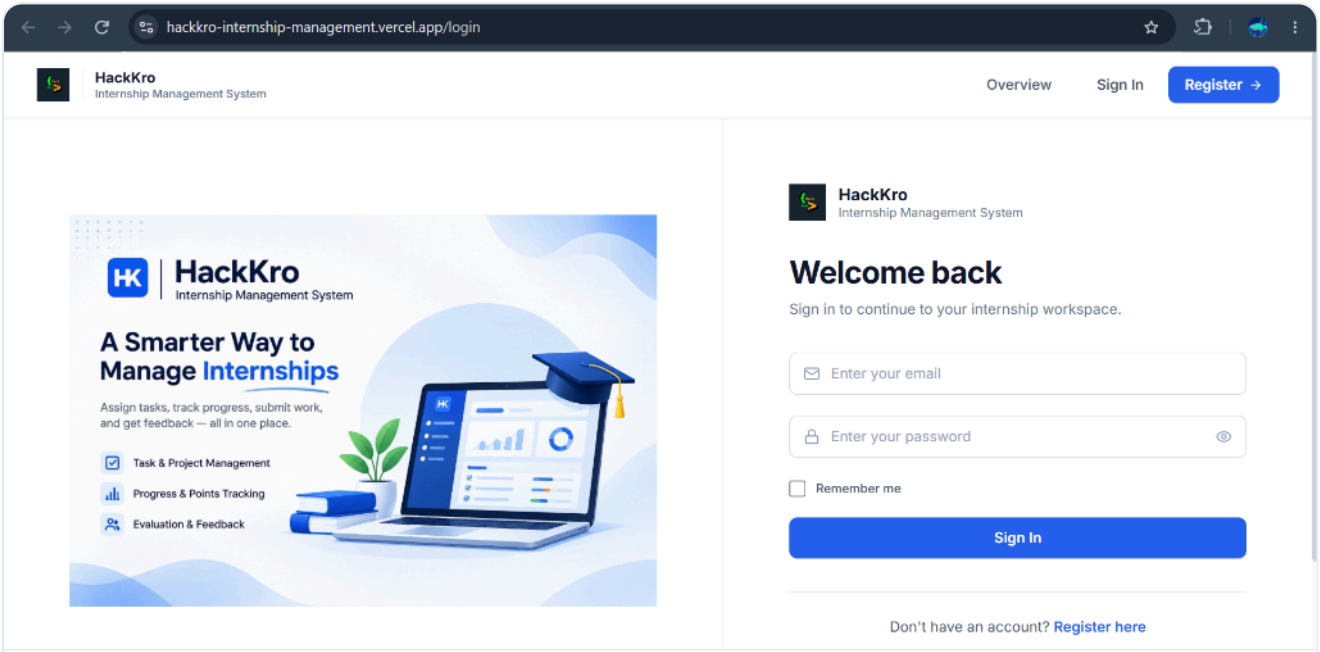


Figure 2: Unified authentication split-screen interface with official HackKro branding illustration.

15. Local Installation & Reproduction Guide

1. **Clone Repository:**

```
git clone https://github.com/neerajcoder1/hackkro-internship-management.git
```

```
cd hackkro-internship-management
```

2. **Install Dependencies:**

```
cd backend && npm install
cd ../frontend && npm install
```

3. **Run Automated Verification Suite:**

```
npm --prefix backend test
```

4. **Execute Production Build:**

```
npm --prefix frontend run build
```

16. Conclusion & Future Roadmap

The HackKro Internship Management System delivers a robust, secure, and production-tested platform ready for enterprise internship cohort management.

- **GitHub Webhook Ingestion:** Ingest candidate pull requests, commit hashes, and automated CI test results directly.
- **Distributed Redis Caching:** Transition in-memory deduplication sets to distributed Redis clusters for horizontal scaling.
- **Automated Plagiarism Detection:** Code similarity analysis comparing cohort submissions against open-source repositories.
- **Certificate Generation:** PDF certificate and performance transcript generation upon internship completion.